The Aether Programming Language — Reference Manual

More with less

Created by Ignacio Alfredo Savi Gualco

2026

Contents

1 Introduction

Aether is a compiled systems programming language targeting x86_64 Linux and x86_64 Windows (AArch64 support is in progress). Programs compile directly to assembly with no runtime and no libc dependency: on Linux the generated code talks to the kernel through raw syscalls, and on Windows through kernel32.

Every code example in this manual has been compiled and executed with the current compiler (`aetherc`) on x86_64 Linux; sections describing behavior on Windows have been verified under the MinGW64 toolchain. Features that exist in the language design but are not yet reliable are explicitly marked **work in progress (WIP)** — they may parse, partially work, or be Linux-only. Nothing in the non-WIP sections is aspirational.

1.1 Hello, world

```
pub func main() -> i32 {
    println("Hello from Aether");
    return 0;
}
```

Output:

```
Hello from Aether
```

1.2 Building and running a program

Linux:

```
./target/release/aetherc hello.ae --arch x86_64 --os linux -o hello.s
bash scripts/assemble_link.sh x86_64-linux hello.s hello
./hello
```

Windows (MSYS2 MINGW64 shell, or the GUI editor's Build & Run):

```
./target/release/aetherc hello.ae --arch x86_64 --os windows -o hello.s
bash scripts/assemble_link.sh x86_64-windows hello.s hello
./hello.exe
```

One-line installers, the multi-tab GUI editor (`tools/aether_editor.py`) and the setup wizard (`tools/aether_setup_gui.py`) are described in the README.

2 Lexical structure

- Source files use the `.ae` extension and UTF-8 encoding.
- Every statement ends with a semicolon `;`.
- Comments: `// line comment` (to end of line).

- Identifiers: letters, digits and `_`, not starting with a digit.
- Keywords include: `pub`, `func`, `return`, `let`, `while`, `break`, `continue`, `if`, `else`, `struct`, `static`, `import`, `try`, `except`, `throw`, `true`, `false`.
- Integer literals: decimal, e.g. `42`, `-17` (unary minus is folded at parse time and keeps the integer type).
- Float literals: must contain a decimal point, e.g. `3.14`, `-0.25`.
- String literals: double-quoted, e.g. `"Hello"`, with `\n`, `\t`, `\"`, `\\` escapes.

3 Types

Type	Description
`i32`	32-bit signed integer
`i64`	64-bit signed integer
`f32`	32-bit float (WIP in some contexts, see below)
`f64`	64-bit float
`String`	UTF-8 string, lowered as a (pointer, length) word pair
`[T; N]`	fixed-size array of `N` elements of type `T`
`vec[T]`	growable vector (heap allocated, explicit free)
`HList`	heterogeneous list of tagged values
`&T`	pointer/reference to a value of type `T`
user types	`struct` definitions, single inheritance

There is no dedicated boolean type in the surface syntax yet (**WIP**); comparisons produce integer `1` (true) or `0` (false) and any integer can be used as a condition (zero is false, nonzero is true). The literals `true` and `false` are reserved.

4 Variables and assignment

Variables are declared with `let`, an explicit type, and an initializer. They are mutable; reassignment must keep the same type.

```
pub func main() -> i32 {
    let x: i64 = 1;
    let y: i32 = 5;
    let z: f64 = 3.14;
    x = x + 1;
    println(x);
    println(y);
    println(z);
    return 0;
}
```

Output:

```
2
5
3.140000
```

Negative values work for both integers and floats:

```
pub func main() -> i32 {
    let f: f32 = -0.25;
    println(f);
    let n: i32 = -42;
    println(n);
```

```
    return 0;
}
```

Output:

```
-0.250000
-42
```

5 Operators

5.1 Arithmetic

`+`, `-`, `*`, `/` on integers (`i32`, `i64`) and floats (`f32`, `f64`). Integer division truncates toward zero. There is no modulo operator yet (**WIP**); use `a - (a / b) * b`:

```
func gcd(a: i64, b: i64) -> i64 {
    return if (b == 0) { a } else { gcd(b, a - (a / b) * b) };
}

pub func main() -> i32 {
    println(gcd(48, 18));
    return 0;
}
```

Output:

```
6
```

5.2 Bitwise (integers only)

`&` (and), `|` (or), `^` (xor), `~` (not), `<<` (shift left), `>>` (arithmetic shift right).

```
pub func main() -> i64 {
    return (12 & 10) + (12 | 10) + (12 ^ 10) + (1 << 4) + (16 >> 2);
}
```

This program exits with status `48` (8 + 14 + 6 + 16 + 4).

5.3 Comparison

`==`, `<`, `<=`, `>`, `>=` produce integer `1` or `0`. `!=` is not available yet (**WIP**); use `(a == b) == 0`.

There are no logical `&&` / `||` / `!` operators yet (**WIP**); combine conditions with nested `if` expressions or integer arithmetic on the `0`/`1` results of comparisons (e.g. `(a > 0) & (b > 0)`).

5.4 Casts

C-style prefix casts convert between numeric types:

```
pub func main() -> i32 {
    let x: i64 = 7;
    let f: f64 = (f64) x;
    println(f);
    let i: i64 = (i64) 2.9;
    println(i);
    return 0;
}
```

Output (float-to-int truncates):

```
7.000000
2
```

6 Control flow

6.1 `if` / `else` (expression form)

`if` is an **expression**: both branches are required and yield a value.

```
pub func main() -> i32 {
    let a: i64 = 3;
    let b: i64 = 5;
    let m: i64 = if (a < b) { a } else { b };
    println(m);
    return 0;
}
```

Output:

```
3
```

A statement-level `if` without a value (and without `else`) is **WIP**; today you express conditional effects with an `if` expression in an assignment or return, or with `while` loops. This composes well with recursion:

```
func collatz_steps(n: i64) -> i64 {
    let steps: i64 = 0;
    let cur: i64 = n;
    while (cur > 1) {
        cur = if ((cur & 1) == 0) { cur / 2 } else { 3 * cur + 1 };
        steps = steps + 1;
    }
    return steps;
}

pub func main() -> i32 {
    println(collatz_steps(27));
    return 0;
}
```

Output:

```
111
```

6.2 `while`, `break`, `continue`

```
pub func main() -> i32 {
    let i: i64 = 0;
    let total: i64 = 0;
    while (i < 100) {
        i = i + 1;
        total = total + i;
    }
    println(total);
    return 0;
}
```

Output:

```
5050
```

`break` exits the innermost loop; `continue` jumps back to the loop condition. Loops may be nested.

7 Functions

Functions are declared with `func` (optionally `pub`), typed parameters, and a return type. The entry point is `pub func main() -> i32`; its return value becomes the process exit status. Functions may be declared in any order — calls to functions defined later in the file work.

```
func square(n: i64) -> i64 {
    return n * n;
}

pub func main() -> i32 {
    println(square(12));
    return 0;
}
```

Output:

```
144
```

7.1 Recursion

Recursion is fully supported, including deep, non-primitive-recursive patterns such as the Ackermann function:

```
func ackermann(m: i64, n: i64) -> i64 {
    return if (m == 0) { n + 1 }
        else { if (n == 0) { ackermann(m - 1, 1) }
        else { ackermann(m - 1, ackermann(m, n - 1)) } };
}

pub func main() -> i32 {
    println(ackermann(3, 2));
    return 0;
}
```

Output:

```
29
```

More verified algorithmic examples (binomial, Catalan, Lucas, tribonacci, integer square root, popcount, digital root, ...) live in `examples/challenges/`; each prints its computed results next to the expected values.

8 Printing and input

8.1 `println`

`println(expr)` prints the value followed by a newline.

- String literals: `println("text");`
- Integer expressions (variables, arithmetic, function calls): `println(fact(10));`
- Float expressions: printed with six fractional digits, e.g. `0.300000`.

```
pub func main() -> i32 {
    let a: f64 = 0.1;
    let b: f64 = 0.2;
```

```
    println(a + b);
    return 0;
}
```

Output:

```
0.300000
```

WIP: `println` of a `String` *variable or parameter* (as opposed to a literal, a call returning `String`, or a static struct field) currently prints an empty line on Linux. Printing strings returned from functions works:

```
func tag() -> String { return "made in Aether"; }

pub func main() -> i32 {
    println(tag());
    return 0;
}
```

Output:

```
made in Aether
```

8.2 `readln` and `to_int`

`readln()` reads one line from standard input as a `String`. `to_int(s)` converts a string to `i64`, validating the entire string (an optional sign followed by decimal digits only); invalid input prints an error and exits with code 1.

```
pub func main() -> i32 {
    println("enter number:");
    println(to_int(readln()));
    return 0;
}
```

With input `42`, the output is:

```
enter number:
42
```

9 Pointers and references

`&T` is a pointer to a value of type `T`. `&x` takes the address of a local variable, `*p` reads through a pointer, and `*p = value;` writes through it. Pointers are plain machine addresses on the stack: taking one allocates no heap memory, so there is nothing to free.

```
func add_one(p: &i64) -> i64 {
    *p = *p + 1;
    return *p;
}

pub func main() -> i32 {
    let x: i64 = 10;
    let p: &i64 = &x;
    println(*p);
    *p = 42;
    println(x);
    println(add_one(&x));
    println(x);
    let f: f64 = 2.5;
```

```
    let q: &f64 = &f;
    *q = *q * 2.0;
    println(f);
    return 0;
}
```

Output (identical on Linux and Windows):

```
10
42
43
43
5.000000
```

`&` currently takes the address of local variables only; pointers to array elements and struct fields are **WIP**.

10 Error handling: `try` / `except` / `throw`

`throw "message";` raises an exception carrying a `String` explaining why. `try { ... } except (e: String) { ... }` catches it; `e` binds the message. An uncaught throw prints `Exception: <message>` and exits with code 1.

```
pub func main() -> i32 {
    println("before try");
    try {
        println("in try");
        throw "division by zero";
        println("unreachable");
    } except (e: String) {
        println("caught:");
        println(e);
    }
    println("after try");
    return 0;
}
```

Output:

```
before try
in try
caught:
division by zero
after try
```

Supported on x86_64 Linux and Windows.

Integer division by zero throws a catchable `"division by zero"` exception:

```
pub func main() -> i32 {
    try {
        let a: i64 = 10;
        let b: i64 = 0;
        let c: i64 = a / b;
        println(c);
    } except (e: String) {
        println("caught");
        println(e);
    }
```

```
    println("after");
    return 0;
}
```

Output:

```
caught
division by zero
after
```

Outside a `try`, division by zero prints `Exception: division by zero` and exits with code 1. Exceptions do not yet propagate across function calls (**WIP**): a throw is caught by a `try` in the same function, otherwise it ends the process.

11 Structs and inheritance

Structs define named, typed fields. Single inheritance uses `:` and lays the parent's fields out first, so a child value can be treated as its parent.

```
pub struct Animal { name: String }
pub struct Dog : Animal { legs: i32 }

static REX: Dog = Dog { name: "Rex", legs: 4 };

pub func main() -> i32 {
    println(REX.name);
    return 0;
}
```

Output:

```
Rex
```

`static` declares a global initialized with a struct literal. Struct literals use `TypeName { field: value, ... }` and may nest.

11.1 Methods

A function named `TypeName_method(self: I64, ...)` can be called with method syntax on a value of that struct type:

```
pub struct Point { x: i32, y: i32 }

static P1: Point = Point { x: 3, y: 4 };

pub func Point_name(self: I64) -> String { return "Point"; }

pub func main() -> i32 {
    println(P1.name());
    return 0;
}
```

Output:

```
Point
```

WIP: printing *numeric* struct fields (e.g. `println(P1.x);`) currently prints nothing on Linux; `String` fields print correctly. Local struct variables support field assignment (`p.x = 5;`), and printing their `String` fields works, but printing their numeric fields is also WIP.

12 Arrays, vectors and heterogeneous lists

12.1 Fixed-size arrays (WIP)

Array declaration and indexing syntax parses and compiles:

```
func main() -> i32 {
    let xs: [i32; 4] = [1,2,3,4];
    let i: i32 = 2;
    println("val:");
    println(xs[i]);
    return 0;
}
```

WIP: printing an indexed element (`println(xs[i]);`) currently prints nothing; the program above outputs only `val:`. Out-of-bounds indexing is designed to print a runtime error and exit with code 1.

12.2 `vec[T]` — growable vectors

Vectors allocate on the heap, grow automatically, and are freed explicitly. `vec_free` returns `1` if it freed the memory and `0` if it was already freed — no hidden memory management.

```
pub func main() -> i32 {
    let v: vec[i64] = vec_new(2);
    println(vec_len(v));
    vec_push(v, 10);
    vec_push(v, 20);
    println(vec_len(v));
    println(vec_pop(v));
    println(vec_len(v));
    println(vec_pop(v));
    println(vec_len(v));
    println(vec_free(v));
    println(vec_free(v));
    return 0;
}
```

Output:

```
0
2
20
1
10
0
1
0
```

WIP: `vec_*` builtins are Linux-only today; the Windows backend does not yet link them.

12.3 `HList` — heterogeneous lists

An `HList` stores values of mixed types, each tagged with a type id (`0` = i64, `1` = f64, `2` = String, `3` = i32, `4` = f32).

```
pub func main() -> i32 {
    let h: HList = hlist_new(4);
    hlist_push(h, 0, 42);
```

```
    hlist_push(h, 3, 100);
    println(hlist_len(h));
    return 0;
}
```

Output:

```
2
```

`hlist_free(h)` frees the list (returns 1 if freed, 0 if already null); `h[index]` reads an element with bounds checking. **WIP:** HList builtins are Linux-only today.

13 Threads

Threading builtins map to native OS threads (raw `clone` on Linux, `CreateThread` on Windows):

- `spawn("func_name", arg: i64) -> i64` — start `pub func name(arg: i64) -> i32` in a new thread, returns a handle
- `join(handle) -> i32` — wait for the worker and return its result
- `destroy(handle) -> i32` — forcibly terminate (1 on success)

```
pub func worker(arg: i64) -> i32 {
    return (i32)arg;
}

pub func main() -> i32 {
    let h1: i64 = spawn("worker", 101);
    let h2: i64 = spawn("worker", 202);
    let r1: i32 = join(h1);
    let r2: i32 = join(h2);
    let s: i32 = r1 + r2;
    let h3: i64 = spawn("worker", 9999);
    let ok: i32 = destroy(h3);
    println("map_reduce");
    return s;
}
```

Prints `map_reduce` and exits with status 303 (note: Linux exit statuses are truncated to 8 bits by the OS, so the observed code is `303 & 255 = 47`).

This works identically on Windows: `spawn` maps to `CreateThread`, `join` waits with `WaitForSingleObject`, returns the worker's exit code from `GetExitCodeThread`, and closes the handle; `destroy` terminates the thread and closes its handle. No thread handles are leaked.

14 Imports

`import "path/to/file.ae";` textually includes another source file. Paths are relative to the importing file.

`lib.ae`:

```
pub func inc(x: i32) -> i32 { return x + 1; }
```

`main.ae`:

```
import "lib.ae";
pub func main() -> i32 { return inc(41); }
```

This program exits with status 42.

15 Built-in math functions

Verified on x86_64 Linux:

```
pub func main() -> i32 {
    println(abs_i64(-5));
    println(min_i64(3, 9));
    println(max_i64(3, 9));
    return 0;
}
```

Output:

```
5
3
9
```

Also available and verified: `abs_i32`, `min_i32`, `max_i32`.

WIP: the float variants (`abs_f64`, `abs_f32`, `min_f64`, `min_f32`, `max_f64`, `max_f32`) exist but printing their results currently produces incorrect output. `sqrt_f64` / `sqrt_f32` and `str_len` exist in the backend but printing their results currently produces incorrect output; treat them as work in progress. File I/O builtins (`file_open`, `file_read`, `file_write`, `file_close`) are designed but not yet usable from surface syntax.

16 Targets, ABI and runtime behavior

- x86_64 Linux: System V AMD64 calling convention; I/O via raw syscalls; no libc; `main` exits through the `exit` syscall.
- x86_64 Windows: Microsoft x64 calling convention (rcx/rdx/r8/r9, 32-byte shadow space); I/O via kernel32 `WriteFile`/`ReadFile`; `main` exits through `ExitProcess`.
- Strings are `(pointer, length)` word pairs; struct `String` fields occupy two 8-byte words.
- Struct layout is parent-first with 8-byte field alignment.
- Runtime errors (invalid `to_int`, out-of-bounds indexing, uncaught `throw`) print a message and exit with code 1.

17 Work in progress

The following are designed but not yet complete. They may parse, be partially functional, or be platform-limited:

- **Statement-level `if`** (without `else`/value): use `if` expressions.
- **`%` modulo, `!=`, logical `&&`/`||`/`!`**: use the workarounds shown in the Operators chapter.
- **Boolean type in surface syntax**: comparisons yield integer 0/1.
- **`println` of `String` variables/parameters** and of **numeric struct fields**: prints an empty line today.
- **`sqrt_f64`/`sqrt_f32`, `str_len`**: results print incorrectly.
- **Array element printing** (`println(xs[i])`): prints nothing today.
- **File I/O builtins**: not yet usable from surface syntax.
- **Pointers to non-locals**: `&` currently takes the address of local variables only (not array elements or struct fields).
- **Windows:** `vec_*`/`hlist_*` builtins.
- **AArch64 Linux backend**: builds for a subset of the language; x86_64 is the reference target. RISC-V is planned.

Everything else in this manual is verified against the compiler in this repository: the examples were compiled with `aetherc`, assembled and linked with `scripts/assemble_link.sh`, executed, and their output compared to the listings shown.